

Audio-Reactive Shader Playground

WebGL visualizer driven by live FFT analysis, with preset morphing, MIDI controller bindings, and hot-reloading GLSL editing.

WebGL	GLSL	Web Audio	React
-------	------	-----------	-------

VERSION	STATUS	LICENSE	TARGET	LATENCY	UPDATED
0.8.3	Open Source	MIT	WebGL 2	~12 ms	Sep 2026

1. Overview

The Audio-Reactive Shader Playground is a live-coding environment for visuals driven by sound. Incoming audio — microphone, line in, or a loaded file — is analysed each frame into frequency bands and higher-level features, which are published to the running fragment shader as uniforms. Editing the GLSL recompiles in place without losing playback state.

It is built for performance use as much as for authoring. Presets store the full shader and uniform state and can be morphed from one to another over a set duration, MIDI controllers bind to any exposed uniform by learn, and compile errors surface as an overlay while the previous working program keeps rendering — a broken edit never blacks out the screen.

2. Signal path

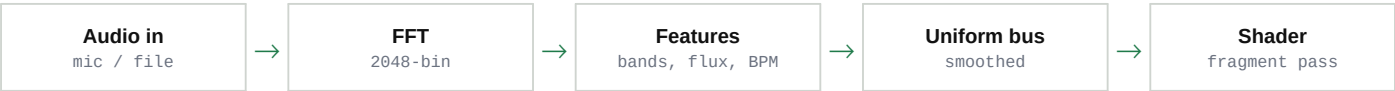


Figure 1 — Analysis runs once per frame on the audio thread's latest buffer; the uniform bus decouples feature rate from render rate.

Raw FFT output is too jittery to drive visuals directly, so every feature passes through a configurable smoother with independent attack and release. A kick drum can hit a uniform instantly and decay over 400 ms, while a spectral centroid used for hue drifts slowly; both come from the same analysis pass.

3. Components

Audio engine	Web Audio graph with AnalyserNode, gain staging, and file or device input switching.
Feature extractor	Band energies, RMS, spectral flux and centroid, onset detection, tempo estimate.
Uniform bus	Named channels with per-channel attack, release, gain, and range mapping.
Shader host	WebGL 2 program cache, hot recompile, error overlay, and last-good fallback.
Preset system	Serialises shader source plus uniform state; interpolates between presets over time.
MIDI layer	Web MIDI learn mode binds any CC to any exposed uniform, saved per preset.

4. Shader interface

```
#version 300 es
precision highp float;

uniform float uTime;           // seconds since start
uniform vec2  uResolution;
uniform float uBands[8];      // smoothed band energies, 0..1
uniform float uRms;           // overall level, 0..1
uniform float uOnset;         // 1.0 on transient, decays by release
uniform float uBpm;           // estimated tempo, 0 when unsure
uniform float uMidi[16];      // learned CC values, 0..1

out vec4 fragColor;

void main() {
    vec2 p = (gl_FragCoord.xy * 2.0 - uResolution) / uResolution.y;
    float r = length(p) - uBands[1] * 0.4 - uOnset * 0.15;
    vec3 col = mix(vec3(0.02), vec3(0.9, 0.3, 0.1), smoothstep(0.02, 0.0, r));
    fragColor = vec4(col * (0.6 + uRms), 1.0);
}
```

Listing 1 — Only uniforms the shader declares are computed; unused features skip extraction.

5. Hot reload behaviour

- **Compile off-thread.** New source compiles into a second program while the current one keeps rendering; the swap happens on a frame boundary, so there is no visible gap.
- **Error surfacing.** GLSL errors map back to editor line numbers and appear as an overlay. The previous working program continues, which is what makes editing safe during a live set.
- **State preservation.** Uniform values, MIDI bindings, and elapsed time survive recompilation, so a shader edited mid-track resumes in phase rather than restarting.

6. Performance

CONFIGURATION	FRAME TIME	AUDIO-TO-VISUAL LATENCY
1080p, single pass	3.1 ms	12 ms
1440p, single pass	5.8 ms	12 ms
1080p, 3-pass feedback	9.4 ms	14 ms

Measured on a laptop RTX 4060 in Chrome 129. Latency is microphone capture to displayed frame, including a 1024-sample analysis hop.

7. Presets and control

- Preset morphing interpolates uniform state and cross-fades between the two compiled programs, which keeps transitions usable even when the shaders are structurally unrelated.
- Bindings are saved per preset, so the same physical controller means different things in different scenes without remapping mid-performance.
- A headless render mode replays a recorded audio file and writes frames at fixed timestep, producing deterministic video rather than a screen capture of a real-time run.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
0.9	Multi-pass graph editor with named buffers and feedback taps.	Q4 2026
0.10	WebGPU backend behind the existing shader host interface.	Q1 2027
1.0	Timeline automation, preset playlists, and stable preset file format.	Q2 2027